

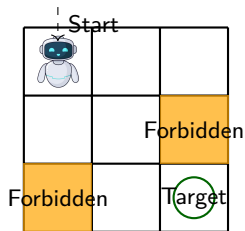
Chapter 1: Basic Concepts

Adapted from Mathematical Foundations of RL by Shiyu Zhao

Outline

- 1 Introduction
- 2 Grid World Example
- 3 State and Action
- 4 State Transition
- 5 Policy
- 6 Reward
- 7 Trajectories, Returns, and Episodes
- 8 Markov Decision Processes
- 9 Summary

Grid World Example



Key Elements:

- **Agent:** Robot that moves in the grid
- **White cells:** Accessible for entry
- **Orange cells:** Forbidden areas
- **Target cell:** Goal to reach

Ultimate Goal:

Find a “good” policy to reach the target without:

- Entering forbidden cells
- Taking unnecessary detours
- Colliding with boundaries

The Learning Challenge

If the agent **knew** the map, planning would be trivial.

The Nontrivial Case:

- Agent has **no prior information** about the environment
- Must interact with the environment to learn
- Find a good policy by **trial and error**

Required Concepts

To solve this problem, we need to understand:

- 1 States and Actions
- 2 State Transitions
- 3 Policies
- 4 Rewards
- 5 Trajectories, Returns, and Episodes

State: Agent's Status

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

Definition:

A **state** describes the agent's status with respect to the environment.

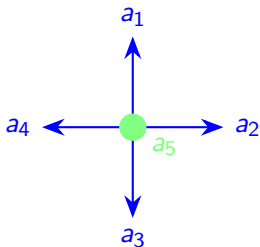
In Grid World:

- State = Agent's location
- 9 cells $\Rightarrow$ 9 states
- States indexed as $s_1, s_2, \dots, s_9$

State Space:

$$\mathcal{S} = \{s_1, s_2, \dots, s_9\}$$

Action: Agent's Choices



Five Possible Actions:

- a_1 : Moving **upward**
- a_2 : Moving **rightward**
- a_3 : Moving **downward**
- a_4 : Moving **leftward**
- a_5 : Staying **still**

Action Space:

$$\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$$

State Transition: Moving Between States

Definition: When taking an action, the agent may move from one state to another.

Example 1: Normal Transition

Agent in state s_1 selects action a_2 (moving rightward):

$$s_1 \xrightarrow{a_2} s_2$$

Example 2: Boundary Collision

Agent in state s_1 takes action a_1 (moving upward):

$$s_1 \xrightarrow{a_1} s_1$$

Agent is **bounced back** — cannot exit the state space.

State Transition: Forbidden Cells

Example 3: Entering Forbidden Cell

Agent in state s_5 takes action a_2 (moving rightward toward forbidden s_6):

Scenario 1: Forbidden cell is **accessible** (but punished)

$$s_5 \xrightarrow{a_2} s_6$$

Scenario 2: Forbidden cell is **not accessible** (e.g., walls)

$$s_5 \xrightarrow{a_2} s_5$$

We consider **Scenario 1**: Forbidden cells are accessible, but stepping into them may get **punished**. This scenario is more general and interesting.

State Transition Table

s_1	s_2	s_3
s_4	s_5	s_6
s_7	s_8	s_9

	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (still)
s_1	s_1	s_2	s_4	s_1	s_1
s_2	s_2	s_3	s_5	s_1	s_2
s_3	s_3	s_3	s_6	s_2	s_3
s_4	s_1	s_5	s_7	s_4	s_4
s_5	s_2	s_6	s_8	s_4	s_5
s_6	s_3	s_6	s_9	s_5	s_6
s_7	s_4	s_8	s_7	s_7	s_7
s_8	s_5	s_9	s_8	s_7	s_8
s_9	s_6	s_9	s_9	s_8	s_9

Each cell indicates the **next state** after taking an action at the corresponding state.

State Transition Probabilities

Mathematical Description: Using conditional probabilities

Example: Transition from s_1 with action a_2

$$p(s_1|s_1, a_2) = 0$$

$$p(s_2|s_1, a_2) = 1$$

$$p(s_3|s_1, a_2) = 0$$

$$p(s_4|s_1, a_2) = 0$$

$$p(s_5|s_1, a_2) = 0$$

Taking a_2 at s_1 will **certainly** cause transition to s_2 .

Stochastic State Transition: Motivation

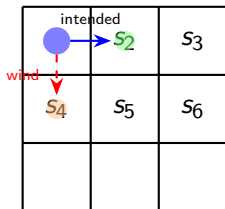
Real-world environments are often uncertain:

- Robot wheels may slip on the floor
- Wind gusts can push the agent off course
- Sensor noise causes imprecise movements
- Mechanical failures lead to unexpected outcomes

Key Difference:

- **Deterministic:** Same action in same state $\Rightarrow$ always same next state
- **Stochastic:** Same action in same state $\Rightarrow$ *distribution* over next states

Stochastic State Transition: Wind Example



Scenario: Random wind gusts

Agent at s_1 takes a_2 (move right):

- **Intended:** Move to s_2
- **Wind effect:** Blown to s_4

The outcome is **probabilistic**, not deterministic.

Stochastic State Transition: Probability Distribution

Example: Taking action a_2 (right) at state s_1 with wind

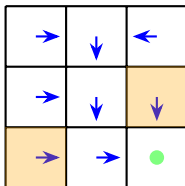
Next State	Probability	Interpretation
s_1	0.1	Blown back by strong headwind
s_2	0.6	Successful movement (intended)
s_3	0	Too far to reach
s_4	0.2	Blown downward by crosswind
s_5	0.1	Blown diagonally

Probability constraint: $\sum_{s' \in \mathcal{S}} p(s'|s_1, a_2) = 0.1 + 0.6 + 0 + 0.2 + 0.1 = 1$

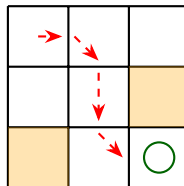
Policy: What Actions to Take

Definition: A **policy** tells the agent which actions to take at every state.

Deterministic Policy



Trajectories from Policy



Deterministic Policy: Mathematical Form

Mathematical Description: Conditional probability distribution $\pi(a|s)$

Example: Policy for state s_1

$$\pi(a_1|s_1) = 0$$

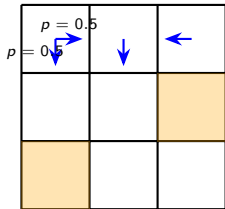
$$\pi(a_2|s_1) = 1 \quad (\text{move rightward with probability 1})$$

$$\pi(a_3|s_1) = 0$$

$$\pi(a_4|s_1) = 0$$

$$\pi(a_5|s_1) = 0$$

This policy is **deterministic**: exactly one action has probability 1.



Example: Stochastic Policy at s_1

$$\pi(a_1|s_1) = 0$$

$$\pi(a_2|s_1) = 0.5 \quad (\text{rightward})$$

$$\pi(a_3|s_1) = 0.5 \quad (\text{downward})$$

$$\pi(a_4|s_1) = 0$$

$$\pi(a_5|s_1) = 0$$

Agent may move rightward **or** downward with equal probabilities.

Policy: Tabular Representation

	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (still)
s_1	0	0.5	0.5	0	0
s_2	0	0	1	0	0
s_3	0	0	0	1	0
s_4	0	1	0	0	0
s_5	0	0	1	0	0
s_6	0	0	1	0	0
s_7	0	1	0	0	0
s_8	0	1	0	0	0
s_9	0	0	0	0	1

Entry (i, j) = probability of taking j -th action at i -th state.

Note: Policies can also be represented as **parameterized functions** (Chapter 8).

Reward: Feedback from Environment

Definition: After executing an action at a state, the agent obtains a **reward** r as feedback.

- Reward is a function: $r(s, a)$
- Value can be positive, negative, or zero
- **Positive reward:** Encourage the action
- **Negative reward:** Discourage the action

Grid World Rewards

- $r_{\text{boundary}} = -1$ (attempting to exit boundary)
- $r_{\text{forbidden}} = -1$ (entering forbidden cell)
- $r_{\text{target}} = +1$ (reaching target state)
- $r_{\text{other}} = 0$ (otherwise)

Reward: Special Case at Target State

Target State s_9 Behavior

The reward process does **not** have to terminate after reaching s_9 .

Example: Different actions at s_9

- Agent takes a_5 (stay still) at s_9 :
 - Next state: s_9
 - Reward: $r_{\text{target}} = +1$
- Agent takes a_2 (move right) at s_9 :
 - Next state: s_9 (boundary collision)
 - Reward: $r_{\text{boundary}} = -1$

Reward: Human-Machine Interface

Interpretation

Reward serves as a **human-machine interface** to guide agent behavior.

With designed rewards:

- Agent tends to **avoid** exiting boundary
- Agent tends to **avoid** stepping into forbidden cells
- Agent tends to **reach** the target

Important Note

Cannot find good policies by simply selecting actions with greatest **immediate** rewards!
Must consider **total reward** in the long run.

Reward Table

	a_1 (up)	a_2 (right)	a_3 (down)	a_4 (left)	a_5 (still)
s_1	r_b	0	0	r_b	0
s_2	r_b	0	0	0	0
s_3	r_b	r_b	r_f	0	0
s_4	0	0	r_f	r_b	0
s_5	0	r_f	0	0	0
s_6	0	r_b	r_t	0	r_f
s_7	0	0	r_b	r_b	r_f
s_8	0	r_t	r_b	r_f	0
s_9	r_f	r_b	r_b	0	r_t

Where: $r_b = r_{\text{boundary}} = -1$, $r_f = r_{\text{forbidden}} = -1$, $r_t = r_{\text{target}} = +1$

Reward: Probabilistic Form

Mathematical Description: Using conditional probabilities $p(r|s, a)$

Example: Reward at s_1 with action a_1

$$p(r = -1|s_1, a_1) = 1, \quad p(r \neq -1|s_1, a_1) = 0$$

Taking a_1 at s_1 yields $r = -1$ with certainty.

Stochastic Rewards

In general, rewards can be **stochastic**.

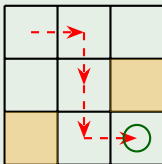
Example: If a student studies hard, they receive positive reward (higher grades), but the specific value may be uncertain.

Trajectory: State-Action-Reward Chain

Definition: A **trajectory** is a sequence of states, actions, and rewards.

Example: Policy 1 Trajectory (from s_1)

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$$



Return: Sum of Rewards

Definition: The **return** is the sum of all rewards along a trajectory.

Example: Policy 1 Return

Trajectory: $s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$

$$\text{return} = 0 + 0 + 0 + 1 = 1$$

Example: Policy 2 Return (passes through forbidden cell)

Trajectory: $s_1 \xrightarrow[r=0]{a_3} s_4 \xrightarrow[r=-1]{a_3} s_7 \xrightarrow[r=0]{a_2} s_8 \xrightarrow[r=+1]{a_2} s_9$

$$\text{return} = 0 - 1 + 0 + 1 = 0$$

Comparison

Policy 1 is **better** than Policy 2 since its return is greater ($1 > 0$).

Immediate vs. Future Rewards

Return = Immediate Reward + Future Rewards

- **Immediate reward:** Reward after taking action at initial state
- **Future rewards:** Rewards obtained after leaving initial state

Key Insight

- Immediate reward may be **negative**
- Future reward may be **positive**
- Decision should be based on **return** (total reward)
- Avoid **short-sighted** decisions!

Discounted Return: Handling Infinite Trajectories

Problem: What if the trajectory is infinitely long?

Example: Infinite Trajectory

$$s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=0]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9 \xrightarrow[r=1]{a_5} s_9 \xrightarrow[r=1]{a_5} s_9 \dots$$

Direct sum: return = $0 + 0 + 0 + 1 + 1 + 1 + \dots = \infty$ (diverges!)

Solution: Introduce **discount rate** $\gamma \in (0, 1)$

$$\text{discounted return} = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1 + \gamma^4 \cdot 1 + \gamma^5 \cdot 1 + \dots$$

$$= \gamma^3(1 + \gamma + \gamma^2 + \dots) = \gamma^3 \cdot \frac{1}{1 - \gamma}$$

Discount Rate: Interpretation

Benefits of Discount Rate $\gamma \in (0, 1)$:

- 1 **Convergence:** Allows for infinitely long trajectories
- 2 **Time preference:** Adjusts emphasis on near vs. far future rewards

γ close to 0:

- More emphasis on **near future**
- **Short-sighted** policy

γ close to 1:

- More emphasis on **far future**
- **Far-sighted** policy
- Dares to take risks for negative immediate rewards

Episode: A Complete Trajectory

Definition: An **episode** (or trial) is a trajectory that ends at a terminal state.

Episode Properties

- Usually assumed to be **finite**
- If environment/policy is stochastic: different episodes from same start
- If deterministic: same episode from same start

Episodic Tasks:

- Have terminal states
- Finite trajectories

Continuing Tasks:

- No terminal states
- Never-ending interaction

Converting Episodic to Continuing Tasks

Method 1: Absorbing States

- Treat terminal state as **absorbing**
- Agent stays forever once reached
- Set $\mathcal{A}(s_9) = \{a_5\}$ or $p(s_9|s_9, a_i) = 1$ for all i

Method 2: Normal State (This Book's Approach)

- Treat terminal state as **normal**
- $\mathcal{A}(s_9) = \{a_1, \dots, a_5\}$
- Agent may leave and come back
- Positive reward every time s_9 is reached
- Agent eventually learns to stay at s_9
- **Requires discount rate** for convergence

Markov Decision Process (MDP): Framework

Definition: An MDP is a general framework for describing stochastic dynamical systems.

Key Ingredients:

Sets

- **State space:** $\mathcal{S}$
- **Action space:** $\mathcal{A}(s)$ for each state $s \in \mathcal{S}$
- **Reward set:** $\mathcal{R}(s, a)$ for each state-action pair

MDP: Model and Policy

Model (Dynamics)

- **State transition probability:** $p(s'|s, a)$
 - Probability of transitioning to s' when taking a in state s
 - $\sum_{s' \in \mathcal{S}} p(s'|s, a) = 1$ for any (s, a)
- **Reward probability:** $p(r|s, a)$
 - Probability of obtaining reward r when taking a in state s
 - $\sum_{r \in \mathcal{R}(s, a)} p(r|s, a) = 1$ for any (s, a)

Policy

- $\pi(a|s)$: Probability of choosing action a in state s
- $\sum_{a \in \mathcal{A}(s)} \pi(a|s) = 1$ for any $s \in \mathcal{S}$

The Markov Property

Definition: The **Markov property** refers to the memoryless property.

Mathematical Form:

$$p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1}|s_t, a_t)$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1}|s_t, a_t)$$

Key Insight

The **next state** and **reward** depend **only** on:

- Current state s_t
- Current action a_t

Independent of all previous states and actions!

Importance: Essential for deriving the Bellman equation (Chapter 2).

Stationary vs. Nonstationary Models

Stationary Model

- Model does **not change** over time
- $p(s'|s, a)$ and $p(r|s, a)$ are time-invariant

Nonstationary Model

- Model may **vary** over time
- Example: Forbidden area may pop up or disappear

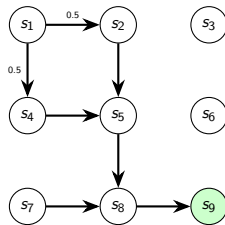
This lecture only considers **stationary models**.

MDP vs. Markov Process

Question: What is the difference between MDP and MP?

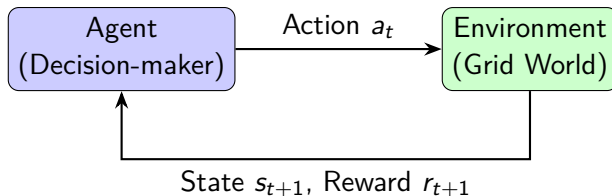
Answer

Once the **policy** in an MDP is **fixed**, the MDP degenerates into a **Markov Process**.



Markov Chain: Discrete-time MP with finite/countable states.

Agent-Environment Interaction



Agent:

- Senses its state
- Maintains policies
- Executes actions

Environment:

- Everything outside the agent
- Returns new state and reward

Chapter Summary

Key Concepts Introduced

- 1 **State** s : Agent's status in environment
- 2 **Action** a : Agent's choices
- 3 **State Transition** $p(s'|s, a)$: Dynamics of environment
- 4 **Policy** $\pi(a|s)$: Agent's strategy
- 5 **Reward** $r(s, a)$: Feedback signal
- 6 **Return**: Cumulative (discounted) rewards
- 7 **Episode**: Complete trajectory
- 8 **MDP**: Mathematical framework
- 9 **Markov Property**: Memoryless property

Next Chapter: Bellman Equation — the fundamental tool for solving MDPs.

Key Equations Summary

State Space: $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$

Action Space: $\mathcal{A}(s) = \{a_1, a_2, \dots, a_m\}$

Transition: $p(s'|s, a), \quad \sum_{s'} p(s'|s, a) = 1$

Policy: $\pi(a|s), \quad \sum_a \pi(a|s) = 1$

Reward: $p(r|s, a), \quad \sum_r p(r|s, a) = 1$

Thank You!

Questions?